Performance Testing

Date	03 October 2026
Team ID	2
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (load testing), psutil via tests/resource_monitor.py (CPU & memory), pytest (functional), server timing logs (real Gemini latency)
Type of Testing	Load testing (50 concurrent users) + real-API latency measurement
Target Module / API	GET /, GET /api/health, GET /nutrition-tip, POST /generate-plan
Test Environment	Local: Windows 11 laptop (8 logical cores, 8 GB RAM, Python 3.13), single Uvicorn worker, SQLite (WAL mode); load test in demo mode, latency test against the live Gemini API
Test Date	03 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Mixed load: home page, health check, nutrition tip and full plan generation + DB write (demo mode)	50 (spawn rate 10/s)	60	Avg < 2 s, max < 5 s, error rate < 1%
2	Concurrent POST /generate-plan writes to SQLite	50	60 (within scenario 1)	No 'database is locked' errors
3	Real Gemini: plan generation, feedback revision, tip (single user)	1	Per request	Valid 7-day plan; latency recorded
4	Primary model overloaded (HTTP 503) → automatic fallback model	1	Per request	Request still succeeds; nothing saved on failure

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	5.4 ms (median 3 ms)	Pass	Application stack (FastAPI + Jinja2 + SQLite), demo mode
2	Response Time (Max)	< 5 seconds	77 ms	Pass	Under 50 concurrent users
3	Throughput (Req/sec)	—	24.5 req/s	Pass	1,457 requests in 60 s
4	Error Rate	< 1%	0% (0 / 1,457)	Pass	WAL mode + busy timeout prevented lock errors
5	CPU Utilization	< 80%	Avg 0.53%, peak 1.74% of system (4.2% / 13.9% of one core)	Pass	Sampled every second with psutil (59 samples)
6	Memory Utilization	< 80%	Avg 81.2 MB, peak 81.6 MB (1.02% of 8 GB)	Pass	Flat after warm-up: no memory leak

Server resource usage (sampled every second during the load test)

Metric	Average	Peak
CPU, % of one core	4.23%	13.9%
CPU, % of whole system (8 cores)	0.53%	1.74%
Memory (RSS)	81.2 MB	81.6 MB
Memory, % of system RAM (8,023 MB)	1.01%	1.02%

Memory figures combine the Uvicorn launcher and server processes on Windows, so the server alone uses slightly less. Raw samples: reports/resource_usage.csv.

Real Gemini API latency (live run)

Operation	gemini-3.8-flash (primary)	gemini-3.5-flash-lite (fallback)	Primary overloaded → fallback
Plan generation	14.3 s	4.3 – 5.4 s	18 – 20 s
Feedback revision	12.8 s	3.8 – 5.5 s	17 – 18 s
Nutrition tip	—	≈ 1 s (7.2 s on first call)	—

Step 4: Observations & Analysis

- The application itself is fast and light: under 50 concurrent users it averaged 5.4 ms (max 77 ms) with zero failures, using under 2% of system CPU and about 82 MB of memory, with no sign of a memory leak. The load test ran in demo mode on purpose, so it measures FastAPI, Jinja2 and SQLite rather than Google's servers, and does not use up the free-tier API quota.

- End-to-end generation time is dominated by the Gemini model: 4–6 s on the light model and 13–15 s on gemini-3.8-flash. The UI shows a loading state during generation.
- During testing Google returned HTTP 503 "high demand" for gemini-3.8-flash on most calls. With one retry only, 4 of 6 requests failed (safely: friendly message, nothing saved). After adding an automatic fallback model, 5 of 5 requests succeeded.
- SQLite WAL mode and a busy timeout were enabled after early concurrent-write tests, giving 0% errors under load.
- Recommendation: for live demos, use the light model as primary for faster, more reliable responses.

Step 5: Screenshots / Evidence

The full Locust HTML report (`reports/perf.html`), CSV statistics (`reports/perf_*.csv`), CPU/memory samples (`reports/resource_usage.csv`), summary (`reports/perf_summary.md`) and server timing logs are in the `reports/` folder at the repository root.

Locust Test Report

During: 10/3/2026, 1:47:14 PM - 10/3/2026, 1:48:14 PM (1 minute)
Target Host: http://127.0.0.1:8000
Script: locustfile.py

Request Statistics

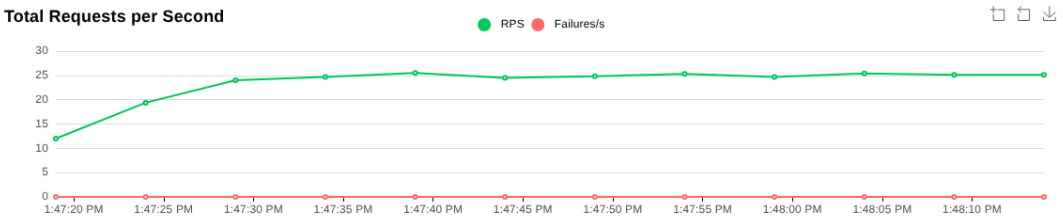
Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /	550	0	3.95	2	63	4947	9.27	0
GET	GET /api/health	353	0	3.97	2	68	180	5.95	0
GET	GET /nutrition-tip	348	0	3.78	1	52	260.23	5.86	0
POST	POST /generate-plan	206	0	14.11	7	77	4643.8	3.47	0
	Aggregated	1457	0	5.35	1	77	2629.77	24.55	0

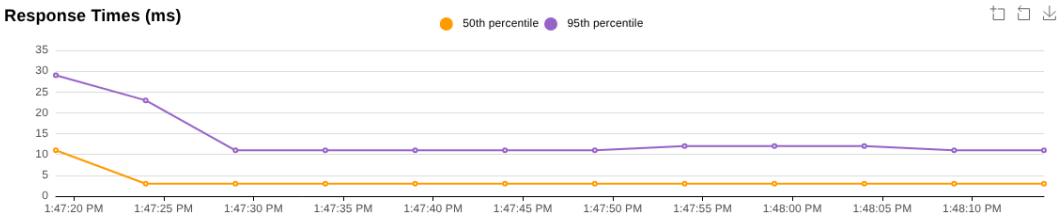
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /	3	3	3	4	6	7	22	63
GET	GET /api/health	3	3	3	4	5	9	21	68
GET	GET /nutrition-tip	3	3	3	4	5	8	28	52
POST	POST /generate-plan	11	11	12	14	25	35	54	77
	Aggregated	3	3	4	7	11	16	42	77

Figure 1: Locust report — request and response-time statistics (50 users, 60 s, 1,457 requests, 0 failures)

Charts





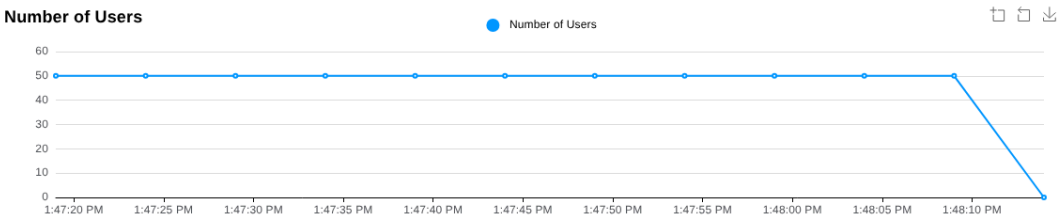


Figure 2: Locust report — requests per second, response-time percentiles and user count